University of London
BSc Computer Science

StudyBuddy: An Iot-Based Environmental Monitoring System For Quantifying Study Space Suitability

Physical Computing and Internet-of-Things (IoT) Final Coursework

Module Code: CM3040

Student Name: Brent Gratias

Student ID: 210108736

Email: bgg5@london.ac.uk

Submission Date: September 14, 2026

Abstract

This report documents the successful implementation, validation, and engineering of "StudyBuddy", a portable, dual-node wireless Internet-of-Things (IoT) environmental monitoring system that quantifies indoor study space suitability.

This system uses two workload-partitioned **ESP8266** microcontroller nodes:

Node 1 (Sensor Node) captures real-time environmental data via four sensors using a range of data transmission protocols, and publishes the readings wirelessly using Message Queuing Telemetry Transport (MQTT) to a public broker.

Node 2 (Display Node) overcomes tight hardware pin constraints on the microcontroller, utilizing daisy-chained **SN74HC595N** shift registers to drive ten red/green parameter status LEDs and a 10-segment LED bar graph, modified into five dual-LED segments to represent sound levels. Node 2 uses an I^2C-driven **LCD1602** scrolling text readout to communicate out-of-range environmental parameters to the user.

Sensor readings are processed locally on Node 2 to calculate a suitability score using a research-based scoring algorithm. It uses triangular curves to score temperature, humidity, and light, and linear step-down curves to penalize rising CO_2 and noise levels. Sub-scores are aggregated into a single 0–100 study suitability score using priority weightings (CO_2: 30%, Noise: 25%, Temp: 20%, Light: 15%, Humidity: 10%) derived from cognitive performance and Indoor Environmental Quality (IEQ) literature.

Both nodes host a local HTTP web server that responds to REST API requests with a JSON payload.

Systematic validation under simulated environmental conditions confirmed high sensor responsiveness, flawless algorithm execution, auto-reconnection reliability, and highly accurate physical-to-digital translation of environmental comfort thresholds.

Table of Contents

Abstract **2**
Table of Contents **3**
List of Figures **5**
List of Tables **6**
Section 1: Introduction and Project Objectives **7**
Context and Problem Statement 7
Project Objectives 7
System Requirements 8
Section 2: System Architecture and Hardware Design **9**
Dual-Node Architecture 9
Hardware Considerations 12
Communication Bus Design 13
Serial-to-Parallel I/O Expansion 16
Section 3: Study Suitability Scoring Algorithm **17**
Section 4: Development Lifecycle and Methodology **22**
Iterative-Incremental Software Development Lifecycle 22
Hardware Hardening: Boot-State Pin Mitigation 22
Calibration Strategy 23
Verification and Validation Strategy 23
Section 5: Network Communication & REST API Dashboard **25**
MQTT Publish-Subscribe Topology 25
Local JSON REST Servers 26
HTTP Dashboard 27
Section 6: Critical Analysis, Reflection, and Future Work **28**

System Strengths and Architectural Successes 28
Engineering Trade-offs and Limitations.....28
Future Roadmap.....29
References 30
Appendices.....32
Appendix A: StudyBuddy List of Components.....33
Appendix B: StudyBuddy Node 1 Schematic 34
Appendix C: StudyBuddy Node 1 Assembled 35
Appendix D: StudyBuddy Node 2 Schematic 36
Appendix E: StudyBuddy Node 2 Assembled.....37
Appendix F: StudyBuddy ESP8266 Pinout Diagram.....38
Appendix G: Project Development Checklist - (in progress) 39
Appendix H: StudyBuddy pseudocode.....40
Appendix I: StudyBuddy Project Development Prototypes 41
Appendix J: StudyBuddy - Web Server Root Path Dashboard.....42

List of Figures

Figure 1: Dual-Node System Architecture Diagram....9

Figure 2: Node 1 - Breadboard layout....10

Figure 3: Node 2 - Breadboard layout....11

Figure 4: Node 1 - Millis( ) timers in main loop( )....12

Figure 5: Node 1 - Four sensor communication protocols....13

Figure 6: Node 1 - getTemperature( ) sensor read error tolerance....14

Figure 7: Node 1 - initializeMAX4466( ) sensor reading midpoint calibration....15

Figure 8: Node 1 - getSound( ) ADC to RMS to Decibel conversion....15

Figure 9: Node 2 - Shift registers to expand microcontroller outputs....16

Figure 10: StudyBuddy Study Suitability Score parameter weighting....17

Figure 11: StudyBuddy Node 2 - Study Suitability Score evaluation constants....18

Figure 12: Graphs showing scoring ranges of temperature, humidity and light using the Bilateral Symmetric Comfort Curve (scoreTriangle) model....19

Figure 13: StudyBuddy Node 2 scoreTriangle( ) function used to calculate suitability score for Bilateral Symmetric Comfort Curve parameters....19

Figure 14: Graph showing scoring ranges of ambient noise using the Unilateral Safe-Plateau Penalty (scoreWarnToMax) model....20

Figure 15: StudyBuddy Node 2 scoreWarnToMax( ) function used to calculate suitability score for Unilateral Safe-Plateau Penalty parameters....20

Figure 16: StudyBuddy Node 2 calculateStudyScore( ) weighting and normalization to a single studySuitabilityScore....21

Figure 17: Message Queuing Telemetry Transport (MQTT) publish subscribe model....25

Figure 18: List of Node 1 REST API Endpoints....26

Figure 19: List of Node 2 REST API Endpoints....27

List of Tables

Table 1: Functional Requirements 8
Table 2: Non-Functional Requirements 8
Table 3: Algorithm Framework and Parameter Thresholds 17
Table 4: Empirical sensor testing 23
Table 5: Functional and Non-Functional Requirements testing 24

Section 1: Introduction and Project Objectives

Context and Problem Statement

Cognitive performance, learning efficiency, and attention span are intrinsically affected by our study environment. Key environmental factors: temperature, humidity, carbon dioxide, lighting, and noise exert a direct influence on concentration and comfort (Wang et al., 2021). Research indicates that poor environmental conditions profoundly degrade learning, memory, and performance on complex cognitive tasks (Zhang *et al.*, 2024).

There is need for a low-cost, low-power, portable system that monitors all five key environmental variables. The system must provide a scientifically grounded scoring mechanism, and instant feedback to clearly communicate local environmental strengths and deficiencies, empowering students to identify and optimize suitable study environments.

Project Objectives

The aim of the StudyBuddy system is to deploy a portable, IoT environmental monitor that quantifies study space suitability for cognitive performance. To achieve this aim, the project executed the following objectives:

Workload-Partitioned Architecture. Design a dual-node system using two **ESP8266** microcontrollers to split data gathering and processing duties. Node 1 (Sensor Node) captures real-time data from environmental sensors, while Node 2 (Display Node) drives the user interface.

Research-Grounded Algorithm. Implement a real-time algorithm using triangular suitability curves (`scoreTriangle()`) for temperature, humidity, and light, and linear safe-plateau falloffs (`scoreWarnToMax()`) for CO_2 and noise. Aggregate scores into a single, intuitive 0-100 Study Suitability Score weighted by cognitive impact.

Output Port Expansion. Overcome microcontroller GPIO limitations by daisy-chaining two **SN74HC595N** 8-bit shift registers. Use this 16-bit parallel interface to drive fifteen status LEDs, alongside a scrolling LCD alphanumeric display.

Node Communication. Link microcontrollers wirelessly using MQTT topology to publish and subscribe to JSON-formatted telemetry. Also deploy HTTP web servers on

both nodes listening on `Port 80` to respond to REST queries with structured JSON payloads enabling external dashboard visualization.

System Requirements

Table 1: Functional Requirements

Req ID	Requirement	Description of Requirement
FR-1	Environmental Data Acquisition	The system must capture ambient temperature, relative humidity, carbon dioxide, ambient light intensity, and acoustic sound levels in real-time.
FR-2	Research-Grounded Scoring Algorithm	The system must aggregate incoming environmental values into a single 0-100 score that grades parameters and penalizes environmental stressors.
FR-3	Identify out-of-range parameters	The system must identify environmental parameters outside of acceptable study suitability levels.
FR-4	Wireless Communication	The system must communicate wirelessly to pass data from the sensor node to the display node.
FR-5	REST API	The system must host a web server, responding with sensor data and suitability score in a structured JSON payload.

Table 2: Non-Functional Requirements

Req ID	Requirement	Description of Requirement
NFR-1	Portability	The system must be compact, USB-powered, standalone.
NFR-2	Useability	The system must make it easy for a user to evaluate and compare study space environments.
NFR-3	Reliability	The system must be reliable even when sensors or communication fails.

Section 2: System Architecture and Hardware Design

Dual-Node Architecture

To overcome the physical performance and GPIO pin limitations of a single microcontroller, the StudyBuddy system adopts a decentralized dual-node architecture (Figure 1). System tasks are partitioned between two **ESP8266** microcontrollers to ensure low-latency responsiveness. Even during intense MQTT network activity or REST API polling, the user interface remains flicker-free.

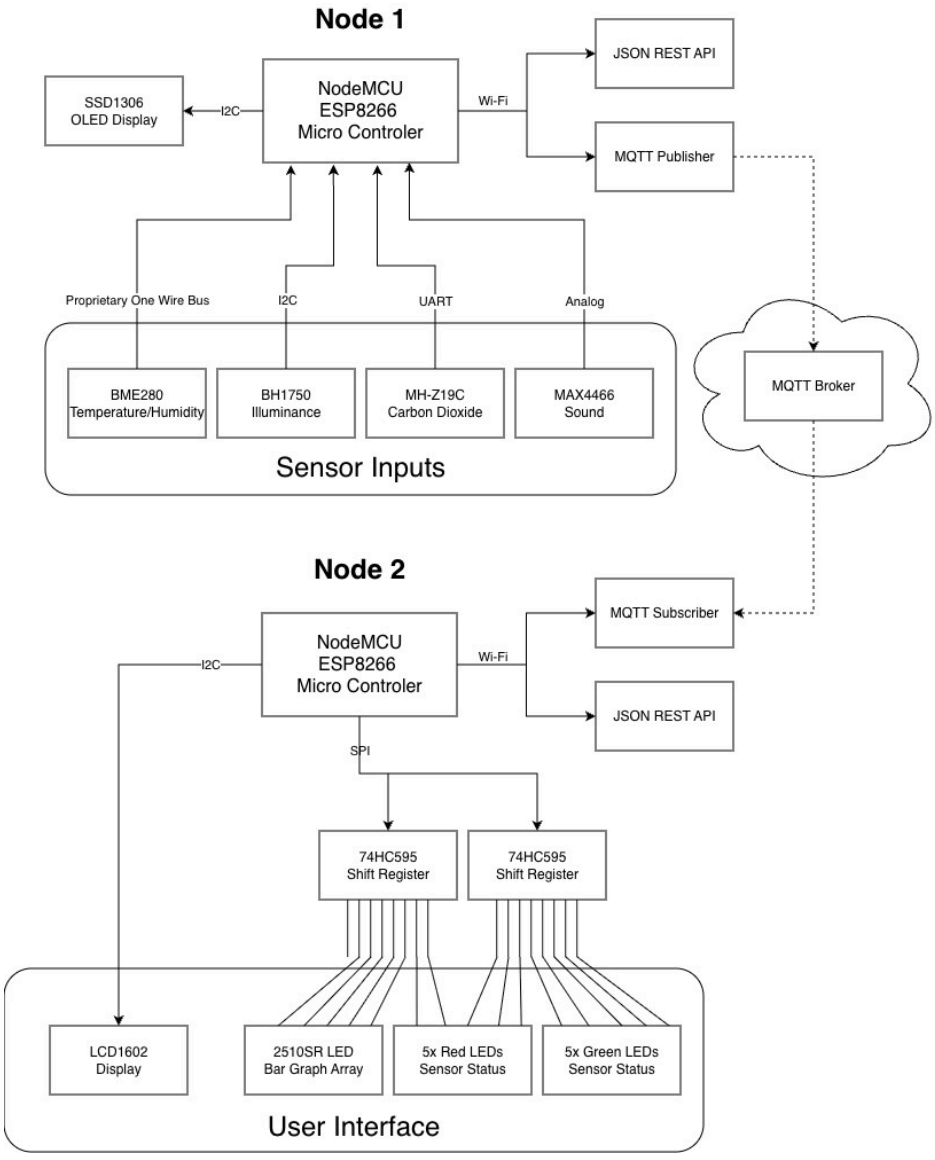


Figure 1: *Dual-Node System Architecture Diagram*

Node 1 (Sensor Node): Is physically located within the study space and serves as the data acquisition transceiver. Real-time environmental data is captured via a multi-sensor suite consisting of:

- **DHT22 (AM2302)** temperature and humidity sensor
- NDIR **MH-Z19C** carbon dioxide sensor
- **BH1750** ambient light meter
- **MAX4466** microphone/amplifier module.

By dedicating Node 1 solely to data acquisition and lightweight local OLED diagnostics, blocking sensor-read latencies do not interfere with user-interface rendering.

Inter-node communication is established via MQTT publish-subscribe topology alongside a concurrent, local HTTP web server that delivers JSON formatted payloads.

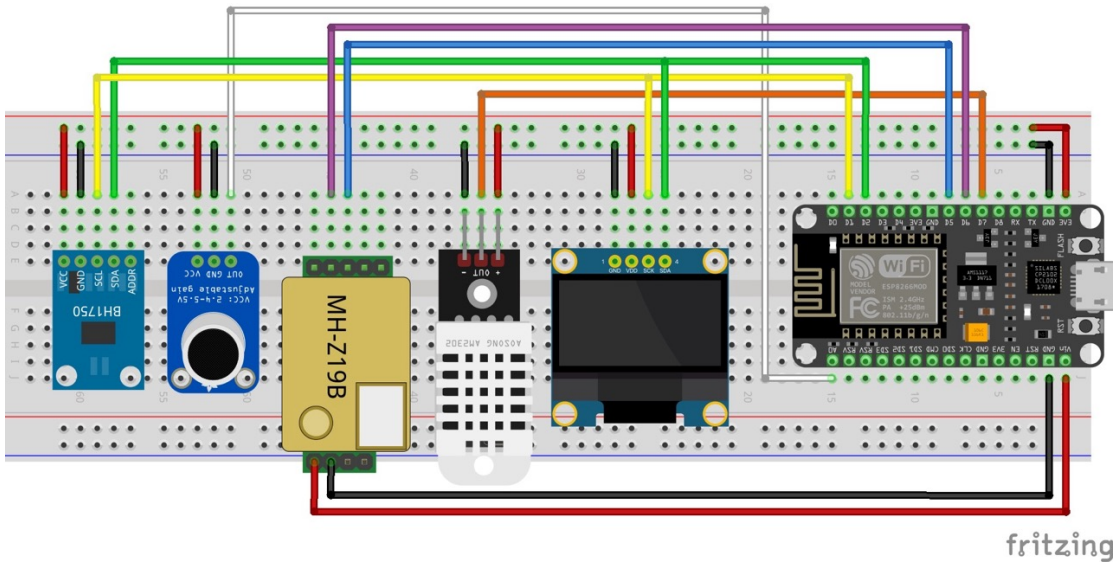


Figure 2: Node 1 - Breadboard layout

Node 2 (Display Node): Acts as the system's primary hub for visual output, mathematical processing, and user interaction. It subscribes wirelessly to Node 1 MQTT sensor telemetry, calculates the suitability algorithm in real-time, and drives an array of LED indicators.

Five Green LED's indicate environmental readings in a suitable study range and five red LED's indicate readings out of range. The 10-segment bar graph (modified to 5 segments) displays ambient sound levels. A 16 x 2 LCD displays the Study Suitability Score and identifies the sensors reading above or below acceptable ranges.

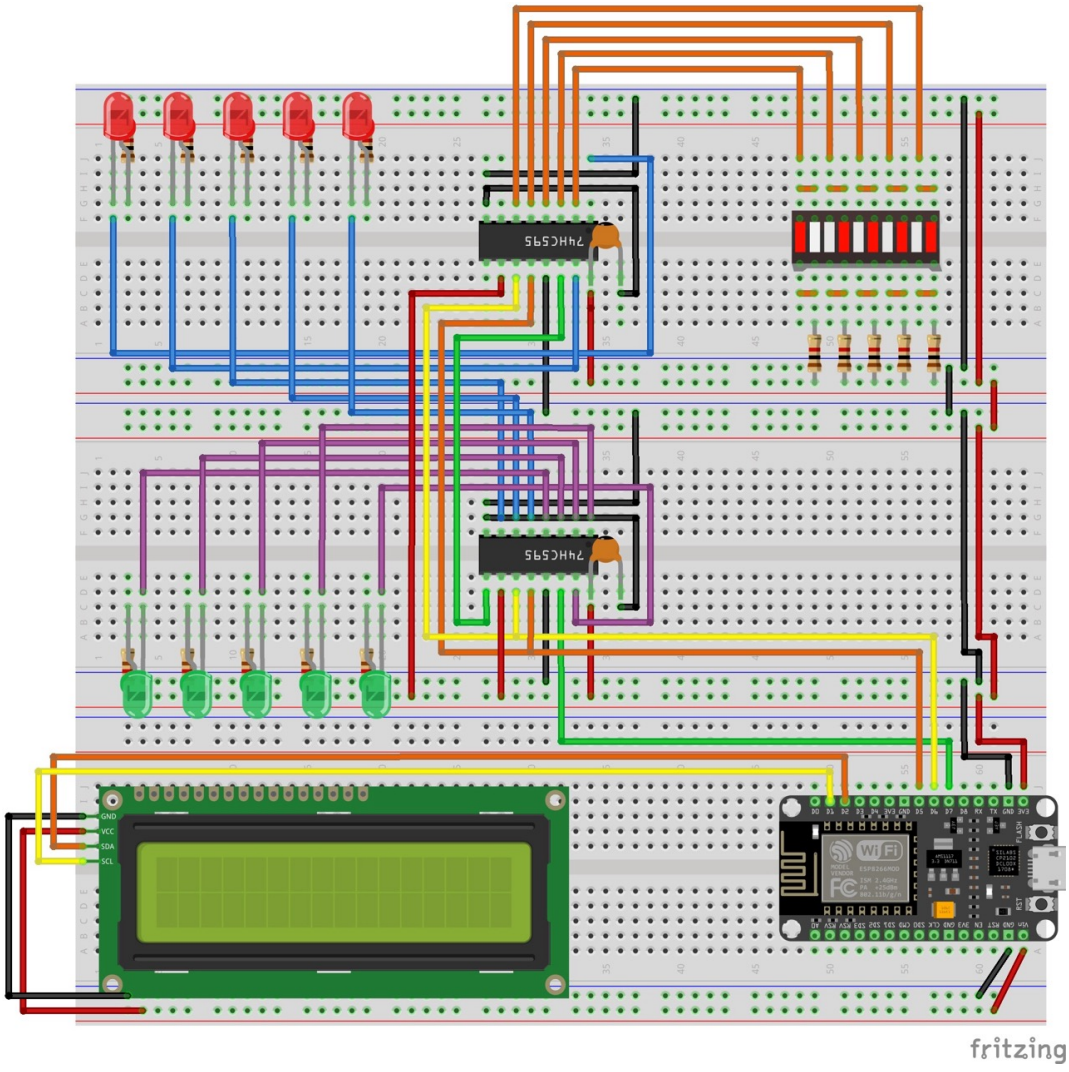


Figure 3: Node 2 - Breadboard layout

Hardware Considerations

Nodes are built around a **Tensilica LX106** 32-bit RISC microprocessor, operating at 80 MHz on an **ESP8266** development board (Espressif Systems, 2020). The processing and memory limitations of this platform present engineering challenges:

Heap fragmentation represents a critical system failure vector. To mitigate this risk, firmware sketches avoid dynamic memory allocation. For example, *`statusMessage String`* array utilizes the *`reserve()`* method to prevent heap fragmentation from continuous string generation.

To ensure the CPU remains dedicated to polling loops without losing network synchronization, automatic modem sleep is deactivated via *`WiFi.setSleepMode (WIFI_NONE_SLEEP)`*.

To reduce CPU load, network traffic, and sensor reads, microcontrollers utilize non-blocking software timers based on the *`millis()`* function as shown in Figure 4. By comparing the elapsed runtime against defined intervals, tasks are executed only when necessary, eliminating redundant processing. Because the **ESP8266** is a single-core processor, *`delay()`* halts execution completely. This would block critical processes such as Wi-Fi handshakes, MQTT packet polling, display scrolling, and web server routing.

```
void loop() {
  // Timing Trackers
  static unsigned long lastSensorTime  = 0;
  static unsigned long lastDisplayTime = 0;

  // Keep alive
  ArduinoOTA.handle();        // watch for OTA requests
  mqtt.loop();                // maintain MQTT connection
  webServer.handleClient();   // watch for Web requests

  // Evaluate timers
  unsigned long now = millis();

  // --- Sensor read & evaluation ---
  if (now - lastSensorTime >= 5000) {
    lastSensorTime = now;
    Serial.println("----------------");
    updateSensorData();
    publishMQTT();
  }

  // --- Display refresh ---
  if (now - lastDisplayTime >= 300) {
    lastDisplayTime = now;
    updateStatusDashboard();
  }
}
```

Figure 4: *Node 1 - Millis( ) timers in main loop( )*

Communication Bus Design

To sample the five distinct microclimate variables, Node 1 integrates four physical communication protocols, balancing bus sharing with electrical isolation.

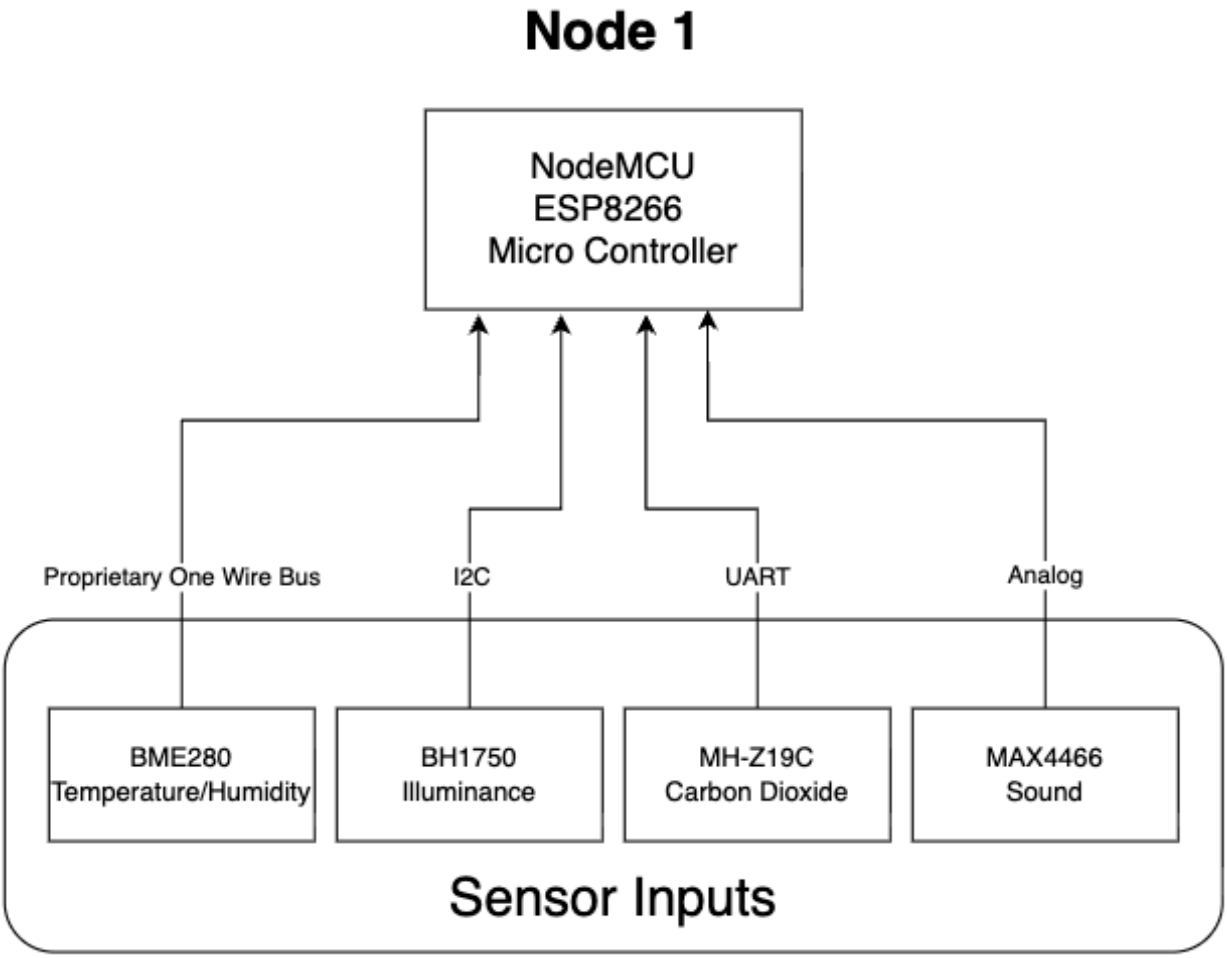


Figure 5: *Node 1 - Four sensor communication protocols*

Proprietary Single-Wire (D7): The **DHT22** temperature/humidity sensor utilizes a single-wire, bidirectional serial bus (Aosong Electronics Co., Ltd., n.d.). To prevent blocking interrupts during the sensor's 40ms handshake, `updateSensorData()` reads are throttled to a conservative 5-second polling interval in `loop()` (Figure 4). Higher **DHT22** sensor reading errors were observed when **ESP8266** WiFi was powered on. To mitigate this, a fault tolerance mechanism was introduced to report the last valid reading for up to `MAX_FAIL_COUNT` times before reporting a sensor failure flag of `0` (Figure 6). This pattern was implemented for the other sensor reads as well.

```
float reading = dht.readTemperature(); // Returns NAN on Error

if (isnan(reading)) {
  failCount++;
  Serial.print("DHT22: Temperature read failed "); Serial.println(failCount);
  if (failCount >= MAX_FAIL_COUNT){
    temperature = 0;
  }
} else {
  Serial.print("DHT22: "); Serial.print(reading, 1); Serial.println(" C");
  temperature = reading;
  failCount = 0;
}
```

Figure 6: Node 1 - getTemperature( ) sensor read error tolerance

I²C Bus (D1/SCL, D2/SDA): The **BH1750** ambient light sensor (address `0x23`) and the **SSD1306** OLED display (address `0x3C`) share a common I²C hardware bus.

Software-Emulated UART (D5/RX, D6/TX): The **MH-Z19C** CO_2 sensor utilizes an asynchronous 9,600 bps serial interface. To prevent bus contention and bootloading failures on the **ESP8266**'s single hardware UART (`GPIO1`/`GPIO3`, shared with the USB-to-UART bridge), this link is established via software UART emulation (`SoftwareSerial`).

Because NDIR optical chambers require thermal stabilization to prevent sensor drift (Zhengzhou Winsen Electronics Technology, 2021), the software runs a non-blocking timer check that suppresses reads during the initial 30-second warmup period.

Analog (A0): The **MAX4466** voice microphone/preamplifier provides an analog signal directly to the **ESP8266**'s single 10-bit analog-to-digital converter (ADC). The signal is AC-coupled in software to eliminate the DC offset which is calculated at sensor initialization (*`initializeMAX4466()`*) (Figure 7). The true root-mean-square (RMS) voltage of the acoustic envelope is calculated and converted to decibels in *`getSound()`* (Figure 8).

```
// Calibration sweep (average of 512 samples)
long sum = 0;
for (int i = 0; i < 512; i++) {
  sum += analogRead(MAX4466_Pin);
  delayMicroseconds(100);
}

// Save the calibrated midpoint (DC Bias) to the global register
dcBias = sum / 512;
```

Figure 7: *Node 1 - initializeMAX4466( ) sensor reading midpoint calibration*

```
// measure RMS of AC-coupled (DC-Bias removed) signal
long sumSquares = 0;
for (int i = 0; i < NUM_SAMPLES; i++) {
  int s = analogRead(MAX4466_Pin) - dcBias;    // get reading and remove DC Bias
  sumSquares += (long)s * s;                   // keep running total of reading squared
  delayMicroseconds(SAMPLE_DELAY_US);
}

double rms = sqrt((double)sumSquares / NUM_SAMPLES); // calculate average RMS

// calculate decibles
double vRms = (rms / 1024.0) * 3.3;            // convert RMS to voltage range
sound = (vRms > 0.0001) ? (20.0 * log10(vRms / V_REF) + 94.0 + CALIBRATION_OFFSET) : 0.0;
```

Figure 8: *Node 1 - getSound( ) ADC to RMS to Decibel conversion*

Serial-to-Parallel I/O Expansion

Driving ten status LEDs and a 10-segment bar graph modified to five dual-LED segments required 15 outputs exceeding the **ESP8266**'s GPIO pins. To resolve this constraint, Node 2 utilizes two daisy-chained 8-bit serial-in, parallel-out shift registers (Figure 9). This allows control of 16 latching outputs using only three pins:

- Data (`SER` / `D7`)
- Shift Clock (`SRCLK` / `D6`)
- Latch Clock (`RCLK` / `D5`)

The 16-bit word is shifted out serially and latched to the output pins by sending an `RCLK` pulse. This latched output design eliminates processor-intensive multiplexing and prevents LED flicker while writing states and during other CPU intensive tasks.

Because the LEDs are wired in active-low common-anode configuration, the 16-bit word is bitwise inverted (`bit = ~bit`), ensuring that writing a logical `1` in software activates the LED.

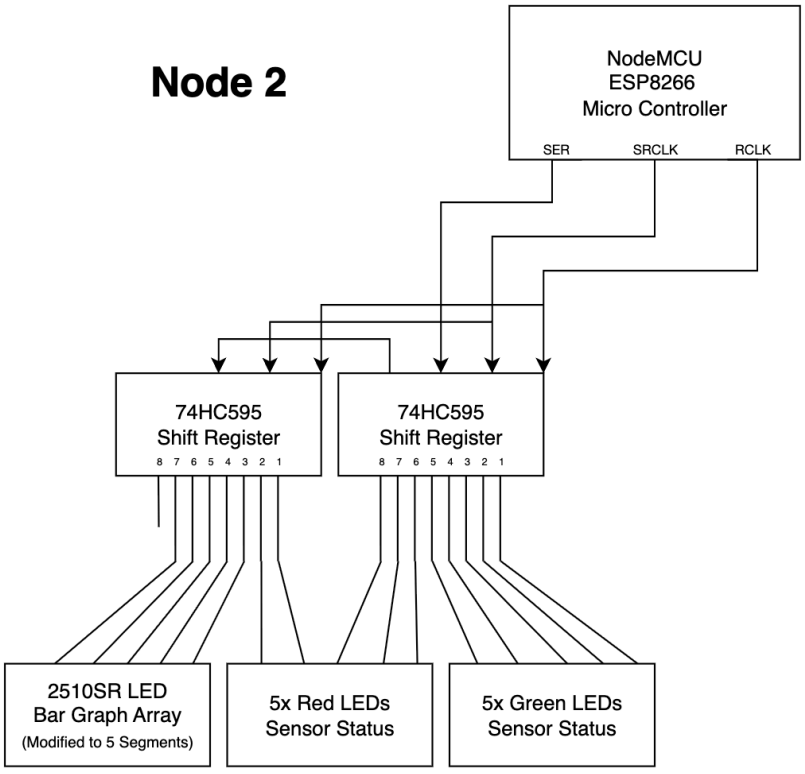


Figure 9: *Node 2 - Shift registers to expand microcontroller outputs*

Section 3: Study Suitability Scoring Algorithm

To calculate a scientifically grounded suitability index, the StudyBuddy scoring engine aggregates environmental data using weighted averages. The hierarchy of parameters directly reflects its documented impact on cognitive performance and learning efficiency.

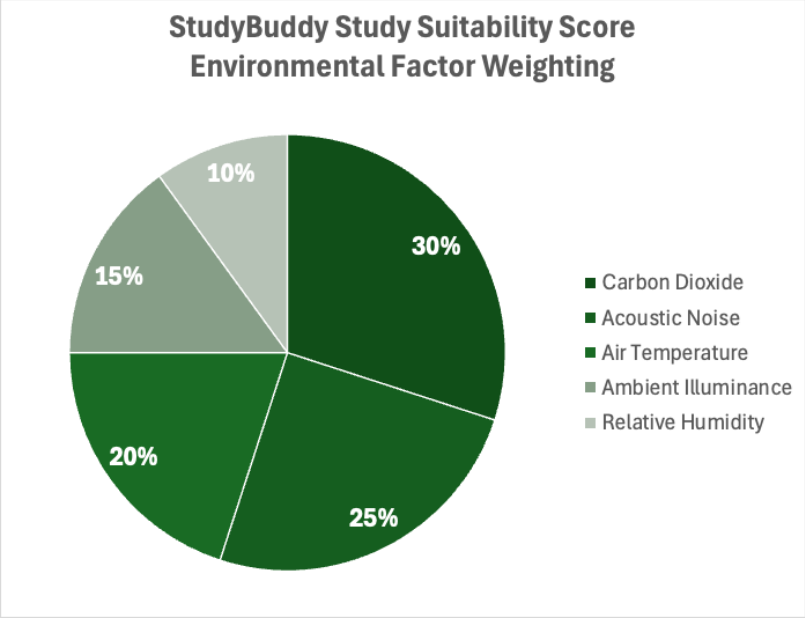


Figure 10: *StudyBuddy Study Suitability Score parameter weighting*

Table 3: Algorithm Framework and Parameter Thresholds

Environmental Parameter	Optimal Range / Warn Limit	Absolute Boundaries (0 Points)	Scoring Method	Priority Weight	Literature Justification
Carbon Dioxide (CO_2)	< 600 ppm (Warn: 600)	> 1,000 ppm	Unilateral Plateau	30%	Elevated levels degrade executive decision-making, concentration, and focus. Sidorin (2015) demonstrated that even moderate CO_2 concentrations exceeding 2,000 ppm double cognitive error rates.
Acoustic Noise	< 40 dB(A) (Warn: 40)	> 65 dB(A)	Unilateral Plateau	25%	Disrupts reading comprehension and short-term working memory encoding. Klatte et al. (2010) proved classroom noise severely impairs speech perception and language comprehension.
Air Temperature	22°C – 25°C	< 15°C or > 30°C	Bilateral Triangle	20%	Governs baseline comfort and productivity; deviations outside the thermal comfort zone of 22°C and 25°C cause measurable reasoning and focus loss (Miri et al., 2025; Wang et al., 2021).
Ambient Light	900 – 1100 lux	< 300 lux or > 1,500 lux	Bilateral Triangle	15%	Impacts visual decoding and cognitive processing speed; daylight levels above 300 lux enhance task accuracy, peaking within an optimal range of 900–1,100 lux (Payedar-Ardakani et al., 2024).
Relative Humidity	40% – 60% RH	< 20% or > 80% RH	Bilateral Triangle	10%	Secondary thermal comfort moderator, altering perceived heat sensation and affecting alertness at extreme levels.

Research backed ranges and priority weightings (Table 3) are declared as immutable software constants on Node 2 (Figure 11).

```
// StudyBudy Suitablility Ranges
const unsigned int  LIGHT_VALUE_MIN     = 300;
const unsigned int  LIGHT_VALUE_MAX     = 1500;
const uint8_t       SOUND_VALUE_WARN    = 40;   // Sound does not have a min
const uint8_t       SOUND_VALUE_MAX     = 65;
const unsigned int  CO2_VALUE_WARN      = 600;  // CO2 does not have a min
const unsigned int  CO2_VALUE_MAX       = 1000;
const float         TEMP_VALUE_MIN      = 15.0;
const float         TEMP_VALUE_MAX      = 30.0;
const uint8_t       HUMID_VALUE_MIN     = 20;
const uint8_t       HUMID_VALUE_MAX     = 80;

// StudyBudy Suitability Score Weighting (adds to 100)
const uint8_t       LIGHT_WEIGHT        = 15;
const uint8_t       SOUND_WEIGHT        = 25;
const uint8_t       CO2_WEIGHT          = 30;
const uint8_t       TEMP_WEIGHT         = 20;
const uint8_t       HUMID_WEIGHT        = 10;
```

Figure 11: *StudyBuddy Node 2 - Study Suitability Score evaluation constants*

The StudyBuddy scoring engine translates sensor readings into normalized 0 to 100 scores using two mathematically distinct methods based on the nature of the environmental variable.

Method 1: Bilateral Symmetric Comfort Curve (*`scoreTriangle()`*)

This method is applied to temperature, humidity, and ambient light where comfort depends on staying within a specific "Goldilocks zone”. The score peaks 100 at the range midpoint and decays symmetrically to 0 at the minimum and maximum boundaries (Figure 12).

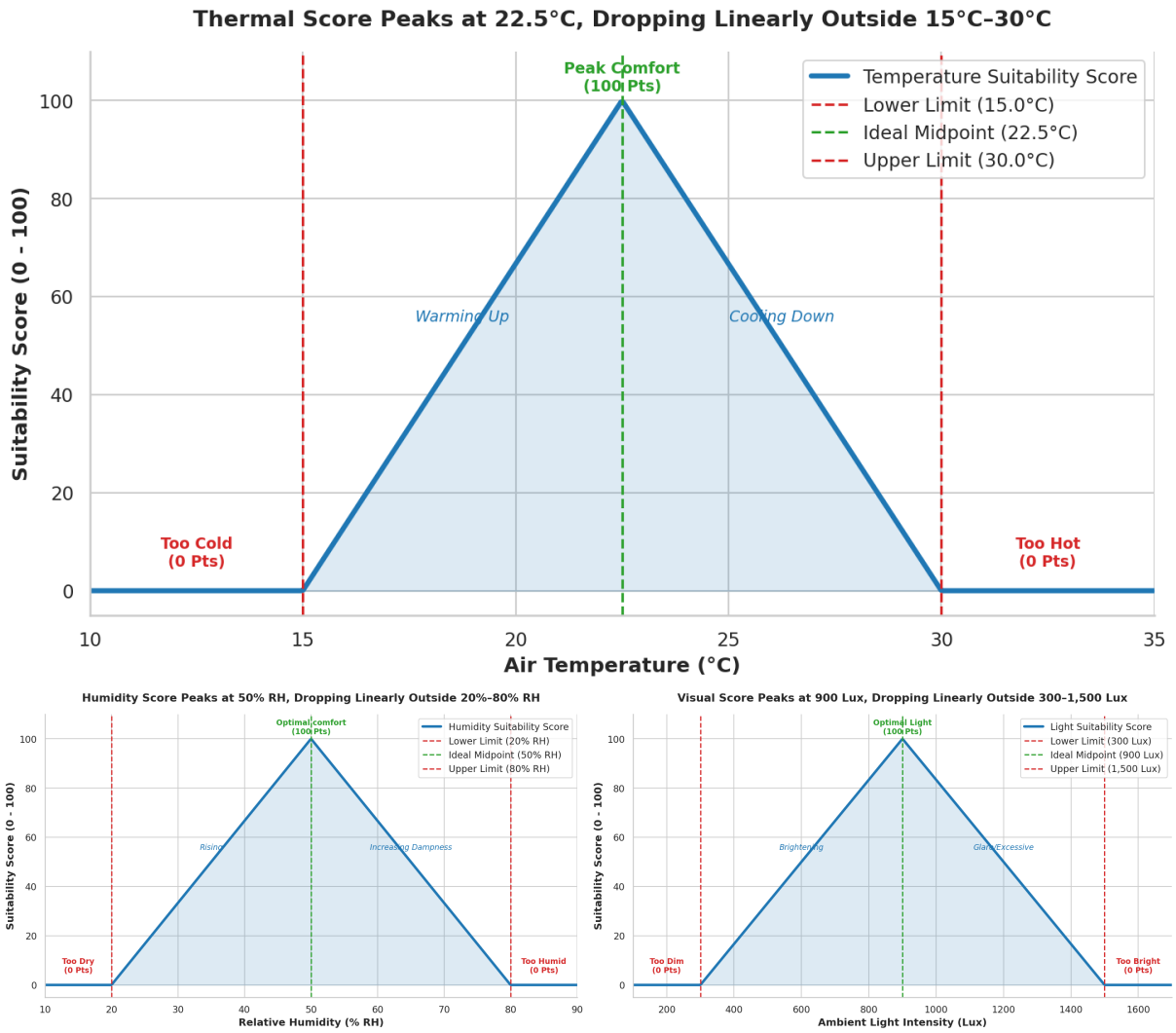


Figure 12: *Graphs showing scoring ranges of temperature, humidity and light using the Bilateral Symmetric Comfort Curve (scoreTriangle) model*

```
// Triangle: 0 at min or below, 100 at midpoint between min/max, 0 at max or above
float scoreTriangle(float value, float minVal, float maxVal) {
  // if the value is out of range, return 0
  if (value <= minVal || value >= maxVal) return 0.0;

  // if the value is in range, calculate distance to center
  float mid = (minVal + maxVal) * 0.5;    // find the mid point
  float dist = fabs(value - mid);         // calculate distance from the middle

  // return the distance from the middle as a 1 to 100 score
  float halfRange = (maxVal - minVal) * 0.5;
  return clampScore(100.0 * (1.0 - (dist / halfRange)));
}
```

Figure 13: *StudyBuddy Node 2 scoreTriangle( ) function used to calculate suitability score for Bilateral Symmetric Comfort Curve parameters*

Method 2: Unilateral Safe-Plateau Penalty (`scoreWarnToMax()`)

This method is applied to environmental stressors where lower levels are always optimal like carbon dioxide and noise.

The score remains 100 as long as readings are below a designated warning threshold. Once the reading exceeds this threshold, the score decays linearly to 0 at the maximum tolerable limit (Figure 14).

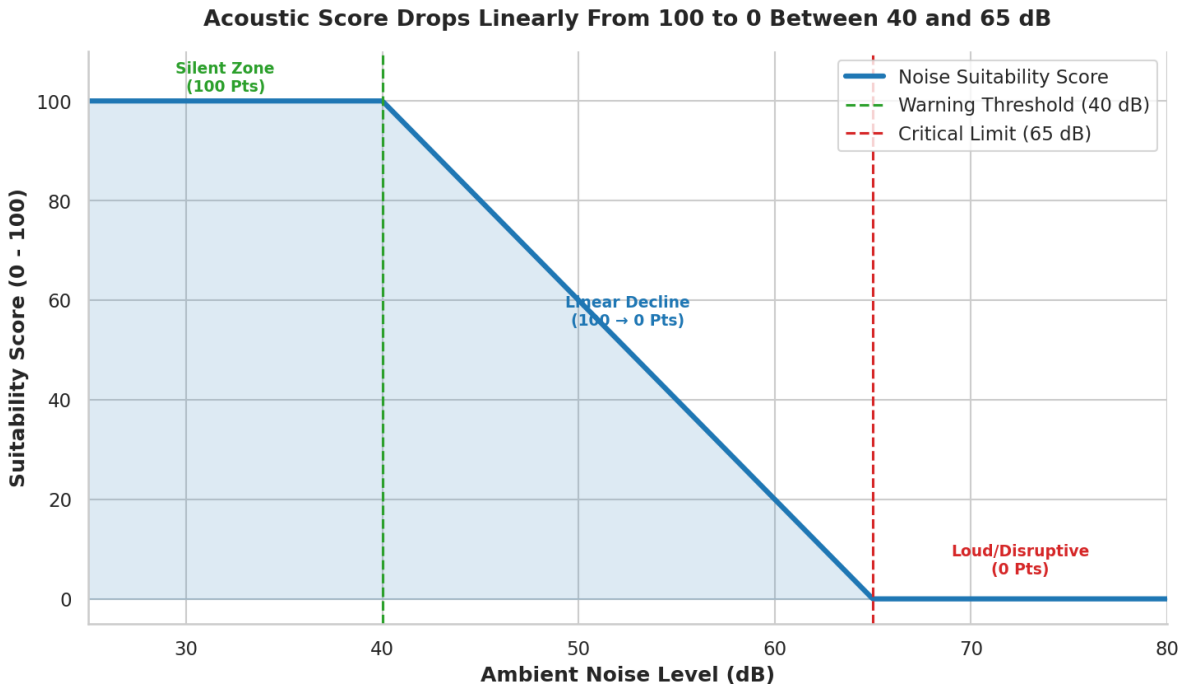


Figure 14: *Graph showing scoring ranges of ambient noise using the Unilateral Safe-Plateau Penalty (scoreWarnToMax) model*

```
// One-sided linear: 100 at warn or below, 0 at max or above
float scoreWarnToMax(float value, float warnVal, float maxVal) {
  if (value >= maxVal || value == 0) return 0.0;
  if (value <= warnVal) return 100.0;

  // calculate the value between warn and max as a normalized value
  float t = (value - warnVal) / (maxVal - warnVal); // 0..1

  // return the normalized value as a 1 to 100 score
  return clampScore(100.0 * (1.0 - t));
}
```

Figure 15: *StudyBuddy Node 2 scoreWarnToMax( ) function used to calculate suitability score for Unilateral Safe-Plateau Penalty parameters*

Individual environmental scores are aggregated by applying research based weightings and normalizing to a single ***studySuitabilityScore*** between 0 and 100 (Figure 16).

```
// Scores are adjusted for weights
float weighted =
    lightScore * LIGHT_WEIGHT +
    soundScore * SOUND_WEIGHT +
    co2Score   * CO2_WEIGHT +
    tempScore  * TEMP_WEIGHT +
    humidScore * HUMID_WEIGHT;

float finalScore = weighted / 100.0; // weights sum to 100

// Final score is clamped to between 0..100
finalScore = clampScore(finalScore);

studySuitabilityScore = (uint8_t)(finalScore + 0.5); // round to int
```

Figure 16: *StudyBuddy Node 2 calculateStudyScore( ) weighting and normalization to a single studySuitabilityScore*

Section 4: Development Lifecycle and Methodology

Iterative-Incremental Software Development Lifecycle

The development of StudyBuddy followed an Iterative-Incremental Software Development Lifecycle (SDLC), transitioning from concept to physical realization through structured task-tracking. A Project Development Checklist was established at the project's onset to track design milestones (Appendix G).

A major developmental pivot occurred following midterm feedback. The initial single-node concept was expanded into a workload-partitioned dual-node architecture. This change was critical because routing four distinct sensor communication buses (I^2C, One-Wire, Software Serial, and Analog ADC) alongside output interfaces on a single microcontroller, although theoretically possible, would have introduced severe timing bottlenecks and bus sharing conflicts.

To manage risk, a modular prototyping strategy was employed. Each sensor was first "sandboxed" on an independent breadboard with an **ESP8266** and an OLED display to isolate code execution and signal integrity (Appendix H). Only once individual sensor subsystems were fully compiled, debugged, and validated were they integrated into the project.

Hardware Hardening: Boot-State Pin Mitigation

During hardware integration, startup failures and sensor dropouts occurred. The Espressif Systems **ESP8266EX** Datasheet revealed that the **ESP8266** requires rigid pin-states at bootup. Connecting active sensor lines or shift-register data pins to these Boot Strapping Pins pulled them into unpredictable states during startup. To harden the physical system, pin layouts were completely redesigned to avoid these pins (Appendix F).

This layout successfully resolved hardware-level conflicts, enabling stable USB firmware uploads and reliable bootstrapping.

Calibration Strategy

To validate **MAX4466** sound sensor readings, a 1Hz sine tone was measured against a smartphone decibel meter. Sensor output consistently read higher than the reference device. A calibration offset of -4.0 dB was hardcoded into the RMS-to-decibel scaling routine.

Testing against a smartphone light meter revealed the **BH1750** ambient light sensor readings consistently 6× lower than reference measurements. Investigation identified the integrated diffuser dome as the cause, with measured transmission loss of approximately 83%. Two solutions were evaluated: (1) removing the diffuser for direct sensor exposure, and (2) applying a software calibration factor. The calibration approach was selected to preserve the diffuser's field-of-view benefits while correcting for the attenuation.

The project migrated from **claws/BH1750** library to **enjoyneering/BH1750FVI** library, which provides built-in optical filter compensation. This allows the sensor to be adjusted based on the diffuser's measured transmission rate, improving accuracy across the full measurement range.

Verification and Validation Strategy

During prototyping, empirical testing was executed to verify sensor responsiveness against standard baselines.

Table 4: Empirical sensor testing

Test ID	Target Req.	Test Objective	Input Action / Physical Simulation	Expected System Result	Actual Observed Result	Status
ETC-01	**Temp**	Verify temperature sensor data acquisition.	Hold sensor in hand to raise temperature.	Temperature reading climbs	Sensor reading climbs on Node 1.	**PASS**
ETC-02	**Humidity**	Verify humidity sensor data acquisition.	Breath on sensor to raise humidity.	Humidity reading climbs	Sensor reading climbs on Node 1.	**PASS**
ETC-03	**CO2**	Verify CO2 sensor data acquisition.	Place sensor in a box with a candle.	CO2 reading climbs	Sensor reading climbs on Node 1.	**PASS**
ETC-04	**Noise**	Verify Sound sensor data acquisition.	Play music in vicinity of sensor.	Sound reading climbs	Sensor reading climbs on Node 1.	**PASS**
ETC-05	**Light**	Verify light sensor data acquisition.	Shine flashlight onto sensor.	Light reading climbs	Sensor reading climbs on Node 1.	**PASS**

On system completion, tests were performed to verify the StudyBuddy system meets the Functional and Non-Functional requirements set out in design.

Table 5: Functional and Non-Functional Requirements testing

Test ID	Target Req.	Test Objective	Input Action / Physical Simulation	Expected System Result	Actual Observed Result	Status
TC-01	**FR-1**	Verify environmental sensor data acquisition.	Apply power to Node 1.	DHT22, BH1750, MH-Z19C, and MAX4466 successfully boot and stream live data within 10s.	All sensors initialized. Local diagnostic OLED displays real-time microclimate values.	**PASS**
TC-02	**FR-2**	Verify piecewise suitability scoring logic.	Introduce physical stimulus (e.g., block light sensor to drop value below 300 lux).	Floating-point score calculation executes: lightScore drops to 0, dragging down the unified studySuitabilityScore.	Suitability score recalculates in real-time. Sub-score decay matches mathematical comfort curves.	**PASS**
TC-03	**FR-3**	Verify parameter status and range alerts.	Exhale directly onto the DHT22 sensor to simulate heavy respiration.	Relative humidity spikes past 80%. Node 2 Humidity LED changes from Green to Red, displays "Humidity too HIGH" on LCD1602.	Out-of-range flag triggered. Red status LED illuminates; LCD begins scrolling localized warning message.	**PASS**
TC-04	**FR-4**	Verify wireless inter-node communication.	Node 1 publishes sensor telemetry JSON package over MQTT.	Node 2 successfully subscribes to studybuddy/node1/sensors, parses JSON, and updates display registers.	Data packets transferred asynchronously over MQTT broker with zero packet loss in standard environment.	**PASS**
TC-05	**FR-5**	Verify REST API compliance on Node 1, port 80	Issue HTTP GET request to *http://<Node1_IP>/sensors*	Local web server responds with a structured, JSON payload containing all five sensor metrics.	HTTP web server routed. Client browser retrieves complete, clean JSON string on Port 80.	**PASS**
TC-06	**FR-5**	Verify REST API compliance on Node 2, port 80	Issue HTTP GET request to *http://<Node2_IP>/sensors*	Local web server responds with a structured, JSON payload containing all five sensor scores and suitability score.	HTTP web server routed. Client browser retrieves complete, clean JSON string on Port 80.	**PASS**
TC-07	**NFR-1**	Verify Portability and low power usage	Run both nodes successfully for 4 hours on 5000mAh portable power pack	After 4 hours of operation both nodes are still operational and power remains on the power pack.	Operated the dual-node system continuously for four hours. At the end of the test, the power bank registered 71% remaining capacity.	**PASS**
TC-08	**NFR-2**	Verify Usability, easy to evaluate one study area as better or worse than another.	Test office and kitchen. Identify which area has a superior study environment.	Expect a clear distinction between study areas. One area will have a higher study suitability score than the other.	Office Study Score: 77 Kitchen Study Score: 61 Higher score clearly identifies the superior study area.	**PASS**
TC-09	**NFR-3**	Verify system reliability and fault tolerance.	Unplug Node 1's physical **I²C** lines to simulate a transient hardware disconnect.	The library registers read failure (reading <= 0). failCount increments; displays retain last valid data up to 5 failed polls.	System filters transient error. If disconnect persists past 10 seconds, error value appears for failed sensor only.	**PASS**

Section 5: Network Communication & REST API Dashboard

MQTT Publish-Subscribe Topology

To pass telemetry wirelessly from Node 1 to Node 2, StudyBuddy utilizes Message Queuing Telemetry Transport (MQTT) architecture over a WiFi network layer.

Node 1 samples its sensors every 5,000 ms using a non-blocking software loop, compiles readings into a JSON string, and publishes the payload to the topic `studybuddy/node1/sensors` via a public broker, hivemq.com, on `Port 1883`.

Node 2 subscribes to this topic from the same broker. It receives the packet, parses the stream using a low-overhead `sscanf()` routine and extracts the five environmental variables directly into global registers with zero heap allocation.

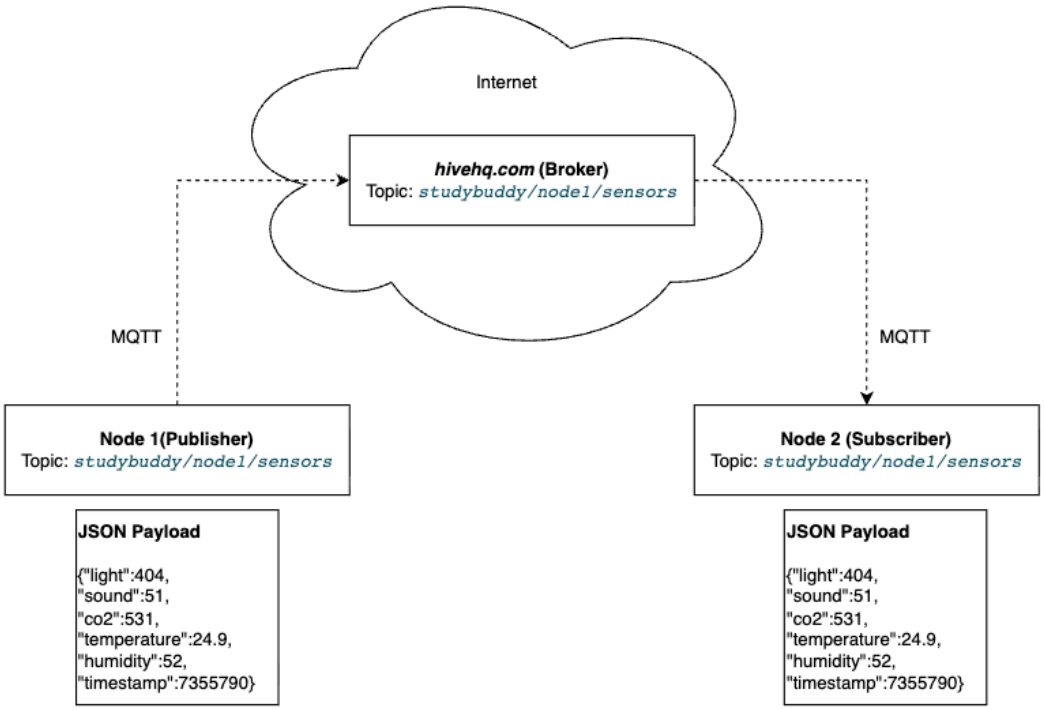


Figure 17: Message Queuing Telemetry Transport (MQTT) publish subscribe model

To satisfy Reliability requirement (NFR-3), a 30-second stale data timer (`DATA_STALE_THRESHOLD = 30000`) runs continuously on Node 2. If wireless packets cease for longer than this limit, the system automatically clears its active

displays and illuminates status indicators to prevent the student from relying on outdated metrics.

Local JSON REST Servers

To provide local diagnostics and support external dashboard integration, both nodes instantiate local HTTP web servers on `Port 80`.

Node 1 Serves as a raw data transceiver. In addition to a master sensor payload at `GET /sensors` it exposes five specialized endpoints to minimize bandwidth consumption when polling individual variables.

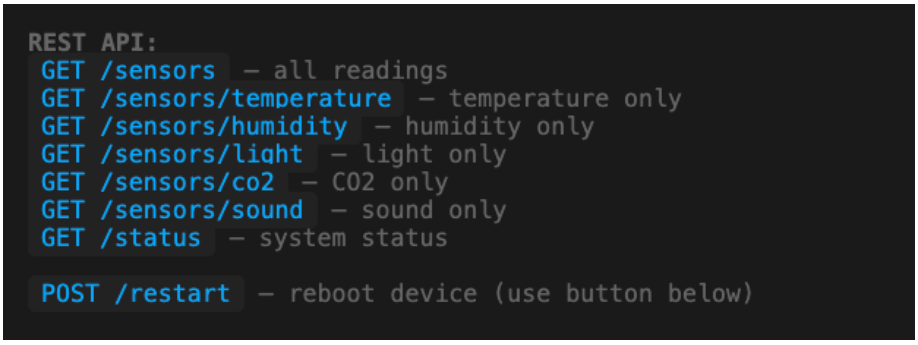


Figure 18: List of Node 1 REST API Endpoints

```
// Example Structured REST Response from Node 1 (GET /sensors)
{
  "light": 410,
  "sound": 43,
  "co2": 527,
  "temperature": 24.7,
  "humidity": 51,
  "timestamp": 1174520
}
```

Node 2 exposes its own version of `GET /sensors`, returning raw sensor values enriched with calculated evaluation attributes, along with the aggregated `studySuitabilityScore`. Other endpoints are also exposed.

Both nodes also expose a `POST /restart` endpoint which triggers a restart of the ESP8266 microcontroller.

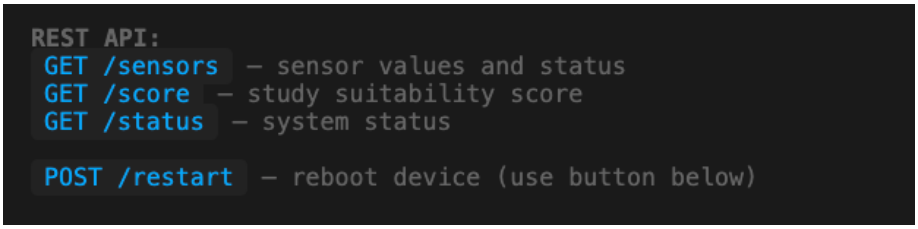


Figure 19: List of Node 2 REST API Endpoints

```
// Example Structured REST Response from Node 2 (GET /sensors)
{
  "light": {"value": 410, "status": "true", "score": 18},
  "sound": {"value": 43, "status": "true", "score": 88},
  "co2": {"value": 527, "status": "true", "score": 100},
  "temperature": {"value": 24.7, "status": "true", "score": 71},
  "humidity": {"value": 51, "status": "true", "score": 97},
  "studySuitabilityScore": 79,
  "statusMessage": "",
  "timestamp": 1174520
}
```

HTTP Dashboard

A custom-styled diagnostic panel is served at the root `/` path of each node. This dashboard provides real-time sensor values, Suitability Scores, out-of-range sensor flags, and network status messages in a browser friendly format. (Appendix J)

Section 6: Critical Analysis, Reflection, and Future Work

System Strengths and Architectural Successes

Several hardware and software engineering decisions helped achieve a robust, responsive, and standalone personal monitoring platform.

The serial-to-parallel **SN74HC595N** shift-register configuration on Node 2 was a major hardware success (Figure 8). By daisy-chaining two 8-bit registers, I successfully expanded the **ESP8266**'s limited GPIO footprint to drive fifteen independent status and bar-graph LEDs using only three digital pins: `D7`, `D6`, and `D5`.

The transition to a fully non-blocking software loop using `millis()` timing registers instead of blocking `delay()` statements was critical a responsive design. By executing sensor sampling and visual updates asynchronously, I was able to make the CPU available to handle MQTT, WebServer requests, and flicker-free display refreshes.

Engineering Trade-offs and Limitations

Several engineering trade-offs were negotiated during the development lifecycle:

While cost-effective, the **ESP8266**'s tight GPIO footprint forced me to implement shift registers and software-emulated UART communications (`SoftwareSerial` for the **MH-Z19C** on pins `D5`/`D6`). I feel an **ESP32**, with its 34+ native GPIO pins, dual-core execution, and multiple hardware UARTS, would have reduced complexity and eliminated software timing overheads.

In network-deprived environments the web dashboard becomes completely inaccessible. The lack of an automatic fallback is a distinct limitation for a device built around portability (NFR-1).

Ranges for temperature and ambient light were hardcoded based on academic research. This represents a usability limitation since comfort is subjective. Students who prefer warmer or dimmer study spaces may desire to customize the target thresholds.

Future Roadmap

It would be interesting to transition StudyBuddy from a breadboard prototype into a commercially viable consumer product.

Connectivity and Integration - Implement an automatic fallback when Wi-Fi connection fails. Expose the parsed JSON states to home automation platforms like Home Assistant or Apple HomeKit via MQTT Discovery to enable automated household responses.

Data and Intelligence - Add trend indicators (e.g., ↑, ↓, →) to indicate short term trends. Integrate data logging to track long-term trends.

Replace the fixed polling interval with an adaptive algorithm to reduce sensor read frequency in stable conditions.

Personalization - Build a responsive web application to allow users to dynamically adjust comfort thresholds and write these directly to the **ESP8266**'s non-volatile EEPROM.

Production - Mount circuitry onto a professionally manufactured, dual-sided Printed Circuit Board (PCB) and house inside a compact enclosure tailored for portable use.

References

Aosong Electronics Co., Ltd. (n.d.) DHT22 (AM2302) Digital-output relative humidity & temperature sensor/module [Datasheet]. Guangzhou: Aosong Electronics Co., Ltd. Available at: https://cdn.sparkfun.com/assets/f/7/d/9/c/DHT22.pdf (Accessed: 8 September 2026).

Espressif Systems (2020) 'ESP8266EX Datasheet', Version 7.1. Shanghai: Espressif Systems. Available at: https://documentation.espressif.com/0a-esp8266ex_datasheet_en.pdf (Accessed: 7 September 2026).

Klatte, M., Lachmann, T. and Meis, M. (2010) 'Effects of noise and reverberation on speech perception and listening comprehension of children and adults in a classroom-like setting', *Noise & Health*, 12(49), pp. 270–282. Available at: https://doi.org/10.4103/1463-1741.70506 (Accessed: 12 July 2026).

Miri, M., Faubel, C., Demarquet Alban, U. and Martinez-Molina, A. (2025) 'Impact of Indoor Environmental Quality on Students' Attention and Relaxation Levels During Lecture-Based Instruction', *Buildings*, 15(16), p. 2813. Available at: https://doi.org/10.3390/buildings15162813 (Accessed: 12 July 2026).

Payedar-Ardakani, P., Gorji-Mahlabani, Y., Ghanbaran, A. H. and Ebrahimpour, R. (2024) 'Daylight illuminance levels, user preferences, and cognitive performance in office environments: Exploring an optimal illuminance range using virtual reality', *Building and Environment*, 258, p. 111638. Available at: https://doi.org/10.1016/j.buildenv.2024.111638 (Accessed: 12 July 2026).

Sidorin, D.I. (2015) 'Dynamics of Carbon Dioxide Concentrations in the Air and Its Effect on the Cognitive Ability of School Students', *Izvestiya, Atmospheric and Oceanic Physics*, 51(8), pp. 871–879. Available at: https://doi.org/10.1134/S000143381508006X (Accessed: 12 July 2026).

Wang, C., Zhang, F., Wang, J., Doyle, J. K., Hancock, P. A., Mak, C. M. and Liu, S. (2021) 'How indoor environmental quality affects occupants' cognitive functions: A systematic review', *Building and Environment*, 193, p. 107647. Available at: https://doi.org/10.1016/j.buildenv.2021.107647 (Accessed: 12 July 2026).

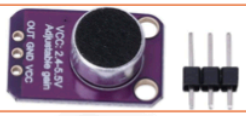
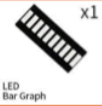
Zhang, N., Liu, C., Li, J., Hou, K., Shi, J. and Gao, W. (2024) 'A comprehensive review of research on indoor cognitive performance using electroencephalogram

technology', *Building and Environment*, 257, p. 111555. Available at: https://doi.org/10.1016/j.buildenv.2024.111555 (Accessed: 10 July 2026).

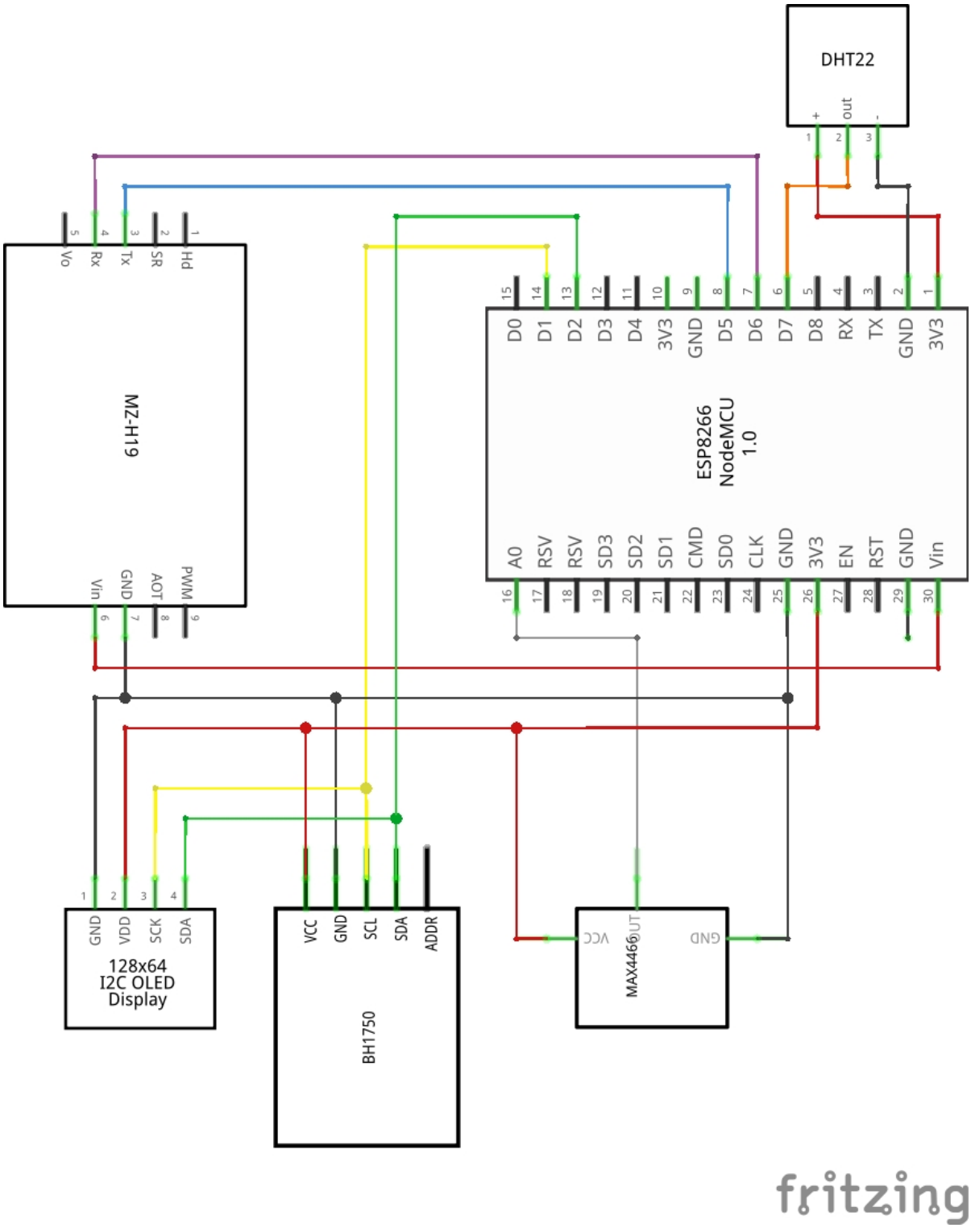
Zhengzhou Winsen Electronics Technology Co., Ltd. (2021) MH-Z19C Intelligent Infrared CO2 Module User's Manual (Version 1.0). Zhengzhou: Winsen Electronics.

Appendices

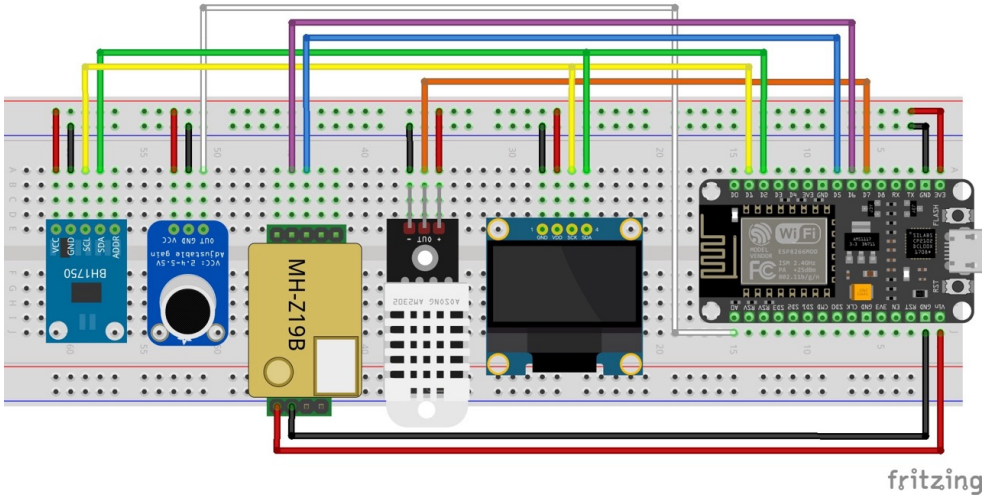
Appendix A: *StudyBuddy List of Components*

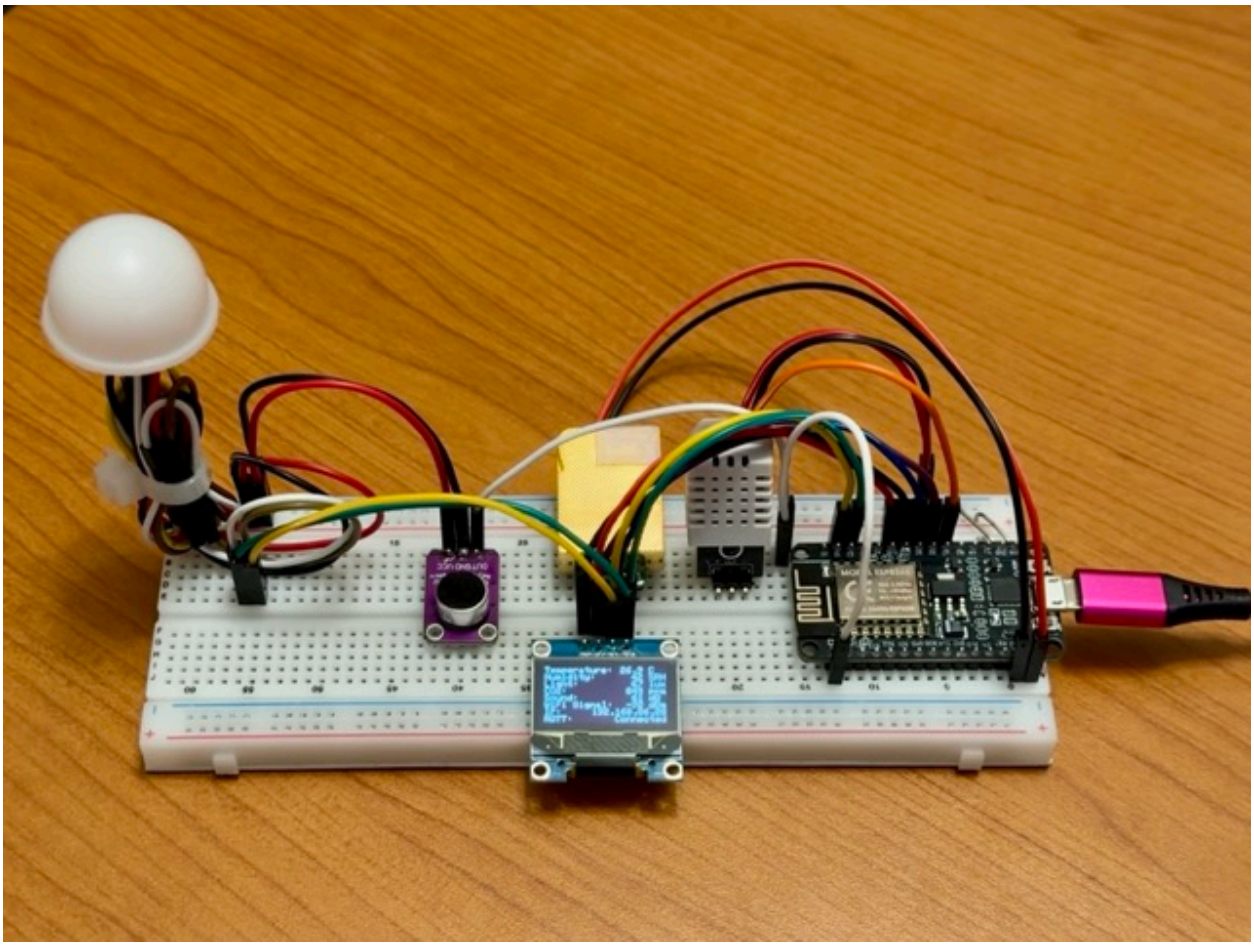
Quant	Model	Description	Interface	Power	Image
1	NodeMCU ESP8266	32-bit 80-160 MHz Tensilica LX106 RISC CPU, 4MB SPI flash memory, 64KB SRAM, AI-Thinker ESP-12E transceiver 2.4 GHz 802.11b/g/n Wi-Fi, AMS1117 LDO regulator, CP2102 USB-to-UART bridge	SPI, I2C, UART	80mA (200mA peak)	
1	BH1750FVI	Environmental Light Intensity Sensor and Module with Difusion Cover	I2C (0x23/0x5C)	2.4V to 3.6V 0.12mA	
1	MH-Z19C	Non-Dispersive Infrared CO2 Gas Sensor Module	UART(9600) / PWM	5V 40mA (150mA peak)	
1	MAX4466	Electret capsule Microphone & Amplifier Module with adjustable gain of 25x to 125x	ANALOG	2.4V to 5.5V DC 0.024 mA peak	
1	DHT22 AM2302	Temperature and Humidity Sensor ±2% RH for humidity, ±0.5°C for temperature	Proprietary Serial	3.3V to 6V DC 0.015mA to 0.060mA (2.5mA peak)	
1	SSD1306	0.96" 128x64 White OLED Display Module with SSD1306 I2C Controller	I2C (0x3C/0x3D)	1.65V to 3.3V ~20mA	
1	LCD1602 I2C	Waveshare LCD1602 I2C Module White color with blue background	I2C (0x7C)	3.3V to 5V 13mA to 26mA	
1	2510SR-1	10-segment red LED bar graph array	N/A	1.8V to 2.0V 20mA peak per segment (10)	
2	SN74HC595N	8-bit serial-in, parallel-out shift register, with 8-bit D-type storage register	SPI	2V to 6V DC 0.08 mA peak	
2	104	0.1 µFd (100nF) Ceramic Disc Capacitor (BypassDecoupling)	N/A	50V rated	
5	Red LED	5mm Standard Difused Red LED	N/A	1.8V 20mA	
5	Green LED	5mm Standard Difused Green LED	N/A	2V 20mA	
10	1K	1K Ohm +/-1% resistors	N/A	.25W	
5	220	220 Ohm +/-1% resistors	N/A	.25W	

Appendix B: *StudyBuddy Node 1 Schematic*

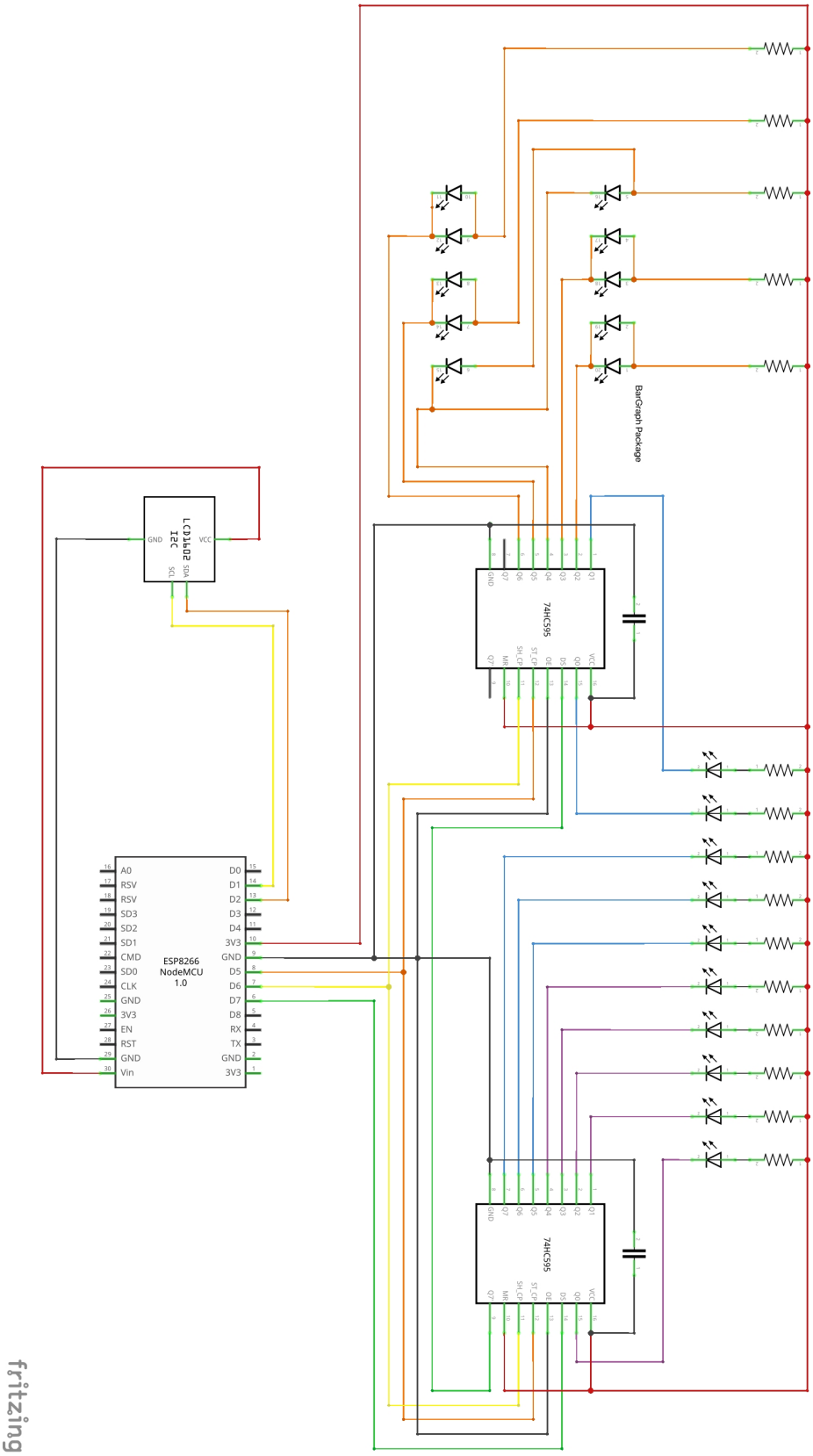


Appendix C: *StudyBuddy Node 1 Assembled*

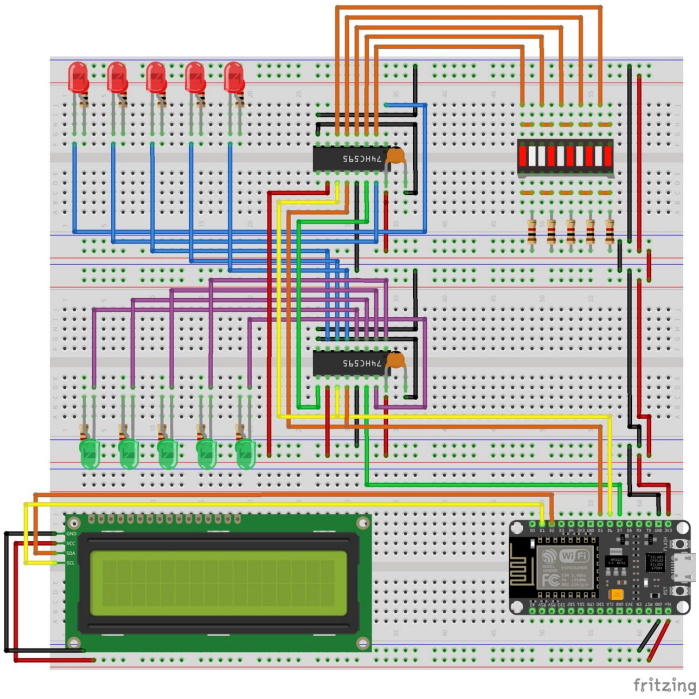


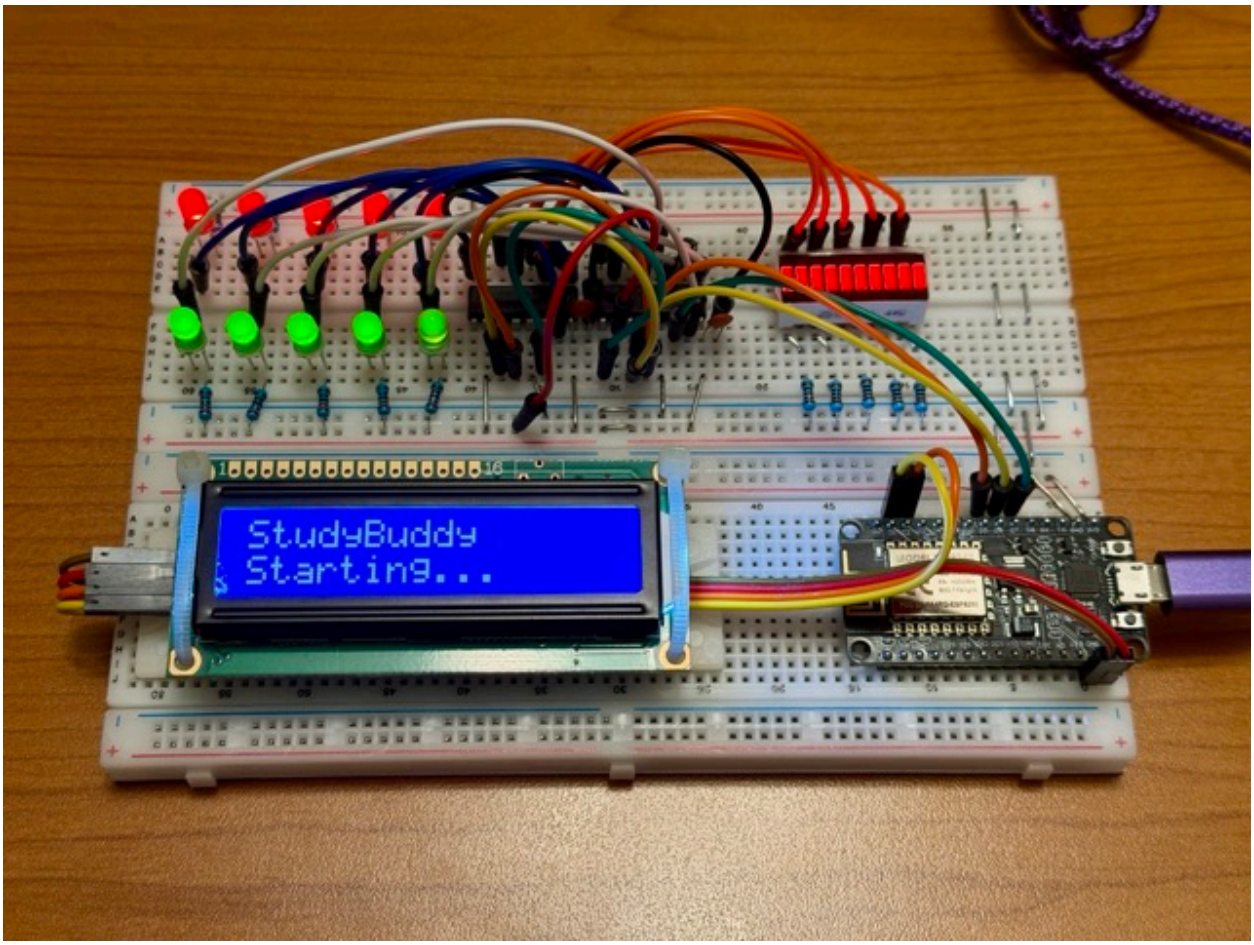


Appendix D: *StudyBuddy Node 2 Schematic*



Appendix E: *StudyBuddy Node 2 Assembled*





Appendix F: StudyBuddy ESP8266 Pinout Diagram

Node 1			
Pin	**GPIO Pin**	**Components**	**Function**
(D0)	GPIO16		*Unused*
(D1)	GPIO5	BH1750, OLED - SCL	I2C SCL
(D2)	GPIO4	BH1750, OLED - SDA	I2C SDA
(D3)	GPIO0		*Unused - Must be HIGH at boot*
(D4)	GPIO2		*Unused - Must be HIGH at boot*
(D5)	GPIO14	MH-Z19C - TX	UART RX(9600)
(D6)	GPIO12	MH-Z19C - RX	UART TX(9600)
(D7)	GPIO13	DHT22 - Out	Single-bus protocol
(D8)	GPIO15		*Unused - Must be LOW at boot*
(A0)	ADC0	MAX4466 - Out	Analog Input (ADC)

Node 2			
Pin	**GPIO Pin**	**Components**	**Function**
(D0)	GPIO16		*Unused*
(D1)	GPIO5	LCD1602 - SCL	I2C SCL
(D2)	GPIO4	LCD1602 - SDA	I2C SDA
(D3)	GPIO0		*Unused - Must be HIGH at boot*
(D4)	GPIO2		*Unused - Must be HIGH at boot*
(D5)	GPIO14	74HC595SN(1,2) pin 12	RCLK - Latch Clock
(D6)	GPIO12	74HC595SN(1,2) pin 11	SRCLK - Shift Register Clock
(D7)	GPIO13	74HC595SN(1) pin 14	*SER - Serial Data input*
(D8)	GPIO15		*Unused - Must be LOW at boot*
(A0)	ADC0		*Unused*

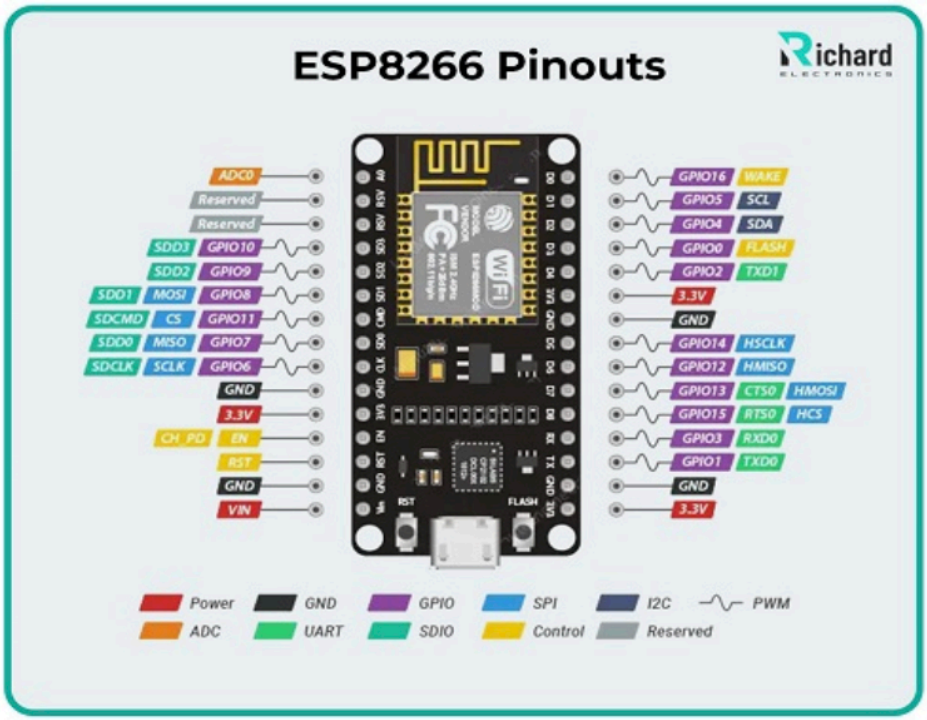


Diagram from Richard Electronics (https://www.richardelectronics.com/)

Appendix G: Project Development Checklist - (in progress)

StudyBudy

Project Development Checklist

CONCEPTUALIZE

- [x] Analyze project requirements (MT)
- [x] Choose concept (MT)
- [x] Research concept (MT)
- [x] Create non-functional requirements chart (MT)
- [x] Sketch system diagram (MT)

DESIGN

- [x] Create ~~physical component list~~ List of Components (MT)
- [x] Source physical components (MT)
- [x] Create system diagram (MT)
- [x] Create pinout diagram (MT)
- [x] Create whiteboard diagram (MT)
- [x] Create electrical schematic (MT)
- [x] Create functional requirements chart (MT)
- [x] Write Pseudocode (MT)

PROTOTYPE

- [x] Build and code prototype each sensor
- [x] Build and code prototype each output
- [x] Build and code prototype node-to-node communication
- [x] Update system diagram
- [x] Update pinout diagram, whiteboard diagram, electrical schematic
- [x] Update functional requirements list
- [ ] Update pseudocode
- [ ] Calibrate Sensors
  - [x] Sound
  - [ ] Light

BUILD & CODE

- [x] Build Node 1
- [x] Code Node 1
  - [x] Scoring
  - [x] Display
- [x] Build Node 2
- [x] Code Node 2
  - [x] No WiFi
  - [x] No MQTT
- [x] Code fault tolerance
- [x] Code OTA firmware updating
- [x] Code Node-to-Node Communication
- [ ] Code REST Servers
- [ ] Code Dashboards

TEST

- [ ] Test node 1 operation
- [ ] Test node 2 operation
- [ ] Test node-to-node communication
- [ ] Test REST API functions
- [ ] Test dashboard functions
- [ ] Test fault tolerance
- [ ] Test non-functional requirements
- [ ] Test functional requirements

DOCUMENT

- [ ] Document compliance with non-functional requirements
- [ ] Document compliance with functional requirements
- [ ] Write hardware operation instructions
- [ ] Write dashboard operation instructions
- [ ] Write REST API interface instructions
- [ ] Create error message chart
- [x] Timings Chart

To Do

- [ ] Light Sensor Too Low?
- [x] Sound Sensor Calabirate
- [ ] Strings to Char.
- [ ] Welcome Screen
- [ ] Define JSON Schema for Rest API

Appendix H: StudyBuddy pseudocode

StudyBuddyPseudocode.txt

```
# Pseudocode for StudyBuddy

# Read all sensors
temperature = readBME280();
humidity = readBME280();
co2 = readMHZ19B();
light = readBH1750();
noise = readMAX4466();

# Score each parameter (0-100) based on optimal ranges
tempScore = scoreTemperature(temperature);    # Optimal: 20-24°C
humScore = scoreHumidity(humidity);           # Optimal: 40-60% RH
co2Score = scoreCO2(co2);                     # Optimal: < 800 ppm
lightScore = scoreLight(light);               # Optimal: 300-500 lux
noiseScore = scoreNoise(noise);               # Optimal: < 40 dB(A)

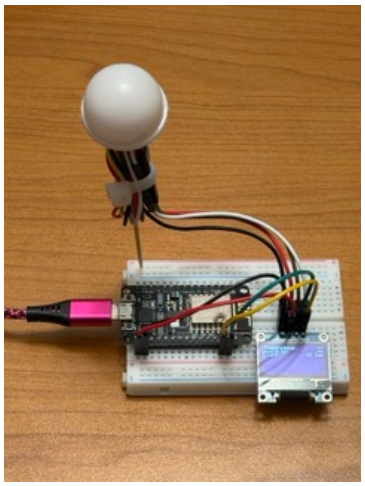
# Calculate weighted suitability score
suitabilityScore =
    (co2Score * 0.30) +
    (noiseScore * 0.25) +
    (tempScore * 0.20) +
    (lightScore * 0.15) +
    (humScore * 0.10);

# Update displays
displayText(temperature, humidity, co2, light, noise);  # 1602 LCD
displayNoiseBar(noise);                                 # 2510SR-1
displayScore(suitabilityScore);                         # 3461BS-1

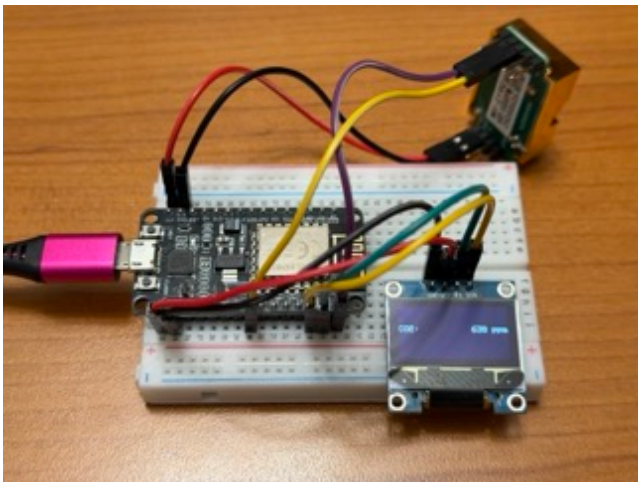
# Update status LEDs (Green/Red per parameter)
updateLED("Temperature", tempScore);
updateLED("Humidity", humScore);
updateLED("CO2", co2Score);
updateLED("Light", lightScore);
updateLED("Sound", noiseScore);

# Serve JSON via REST API
serveJSON(temperature, humidity, co2, light, noise, suitabilityScore);
```

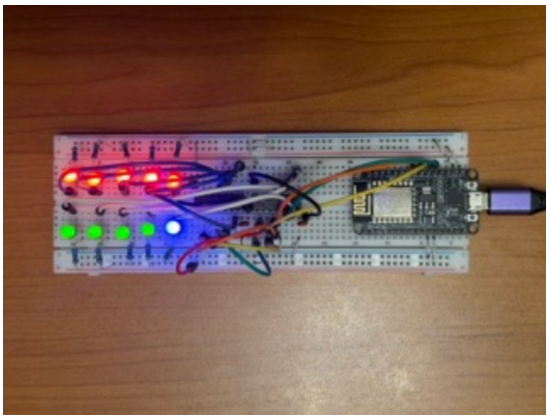
Appendix I: StudyBuddy Project Development Prototypes



BH1750 Sensor Prototype



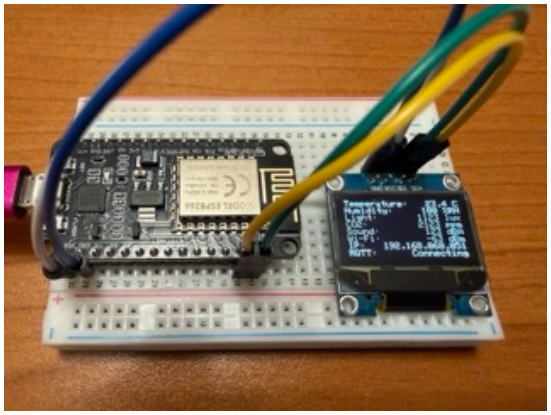
MHZ19B Sensor Prototype



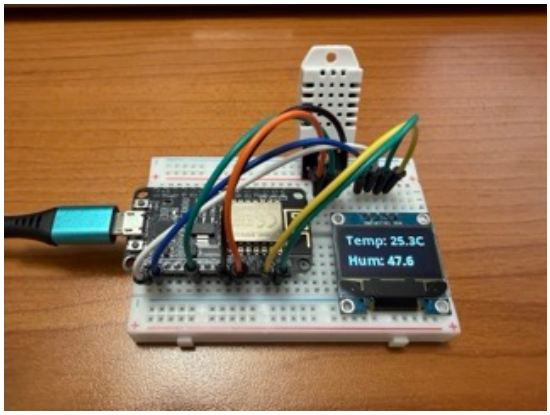
Shift Registers Prototype



MAX4466 Sensor Prototype



SSD1306 OLED Prototype



DHT22 Sensor Prototype

Appendix J: StudyBuddy - Web Server Root Path Dashboard

